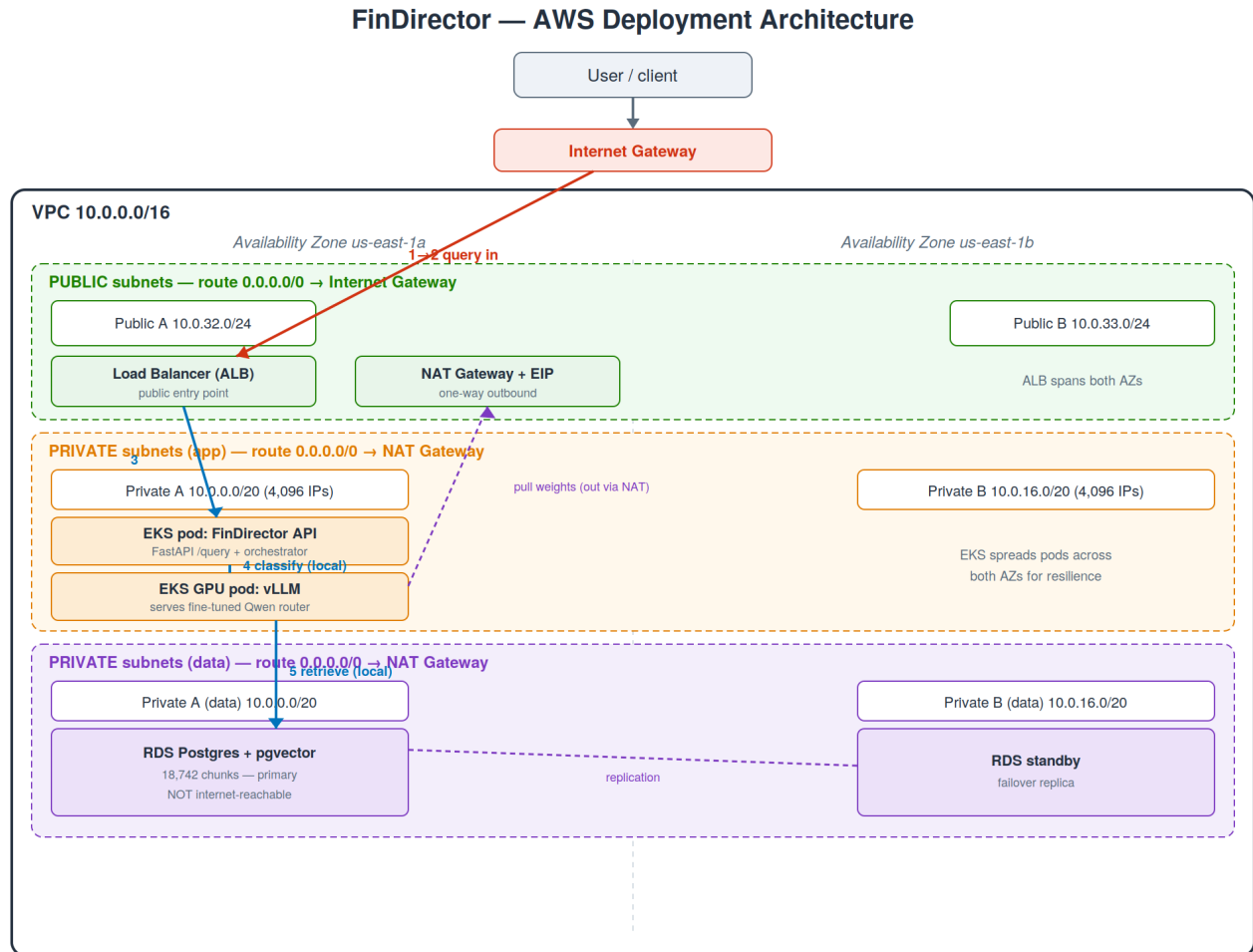


FinDirector — AWS Architecture

The target production deployment: a directive-driven RAG system served from a private, multi-AZ VPC, with the database and compute isolated from the internet and reachable only through a public load balancer. The application (router → orchestrator → retrieval → generation → /query API) is built and runs locally today; this describes how it maps onto AWS.



Two availability zones are shown; each private/public tier has one subnet per AZ. Numbered arrows trace a query in (1→5); the dashed purple arrow is outbound model-weight pull via the NAT.

IP Address Plan

The VPC's 10.0.0.0/16 (65,536 addresses) is carved into four subnets across two AZs. Private subnets are sized large (/20) because EKS assigns an IP per pod and can consume hundreds; public subnets are small (/24) because only a load balancer and NAT live there. Ranges do not overlap; space above 10.0.34.0 is left free for future subnets.

Subnet	AZ	CIDR	Addresses	Tier	Holds
Private A	us-east-1a	10.0.0.0/20	4,096	app + data	EKS pods, vLLM, RDS primary
Private B	us-east-1b	10.0.16.0/20	4,096	app + data	EKS pods, RDS standby
Public A	us-east-1a	10.0.32.0/24	256	internet-facing	Load balancer, NAT gateway
Public B	us-east-1b	10.0.33.0/24	256	internet-facing	Load balancer (spans AZs)

Free for future use: 10.0.34.0 – 10.0.255.255 (e.g. a third AZ, a separate DB tier, or an isolated subnet).

How Routing Makes a Subnet Public or Private

Every subnet has a route table — a lookup list deciding where traffic goes by destination. **10.0.0.0/16** means "any address inside my VPC"; **0.0.0.0/0** means "anywhere" (the catch-all, matched only when no more-specific rule applies). A packet uses the most specific matching rule.

Route rule	Public subnet	Private subnet
10.0.0.0/16 → local	deliver inside the VPC	deliver inside the VPC
0.0.0.0/0 → ?	Internet Gateway (two-way)	NAT Gateway (outbound only)

Both tiers keep internal traffic local. The only difference is where "everything else" goes — the Internet Gateway (reachable from outside) or the NAT (can reach out, cannot be reached in). That single line is the entire public-vs-private distinction.

Traffic Flow 1 — a user query comes IN

1. **User → Internet Gateway.** The request arrives from the public internet at the VPC's only front door.
2. **IGW → Load Balancer.** The ALB sits in a public subnet because it must be internet-reachable — the single public entry point.
3. **Load Balancer → API pod.** Forwarded inward to the FinDirector API pod in a private subnet, over internal VPC routing. The API is not directly internet-reachable; only the ALB can reach it.
4. **API → vLLM pod.** The orchestrator calls the directive router (vLLM GPU pod) to classify — internal VPC traffic, never the internet.
5. **API → RDS.** Retrieval queries RDS Postgres + pgvector — also internal. The database is never exposed; only in-VPC resources can reach it.
6. **Response returns** the reverse path: RDS → API → generation → ALB → IGW → user.

Steps 3–5 all travel the 10.0.0.0/16 → local route — traffic between the system's own components stays inside the VPC and never touches the NAT or internet.

Traffic Flow 2 — a private resource pulls something OUT

1. **vLLM pod → external host** (matches 0.0.0.0/0, not the local range) to download model weights.
2. **Private route table sends 0.0.0.0/0 → NAT Gateway.** Traffic goes to the NAT in the public subnet.
3. **NAT → Internet Gateway → internet.** The NAT makes the request on the pod's behalf, presenting its Elastic IP; the response returns through the NAT.
4. **One-way only.** External hosts see the NAT's IP, not the pod. Nothing on the internet can initiate a connection inward. The pod reached out; the internet cannot reach in.

The same outbound-via-NAT path serves RDS updates, EKS pulling container images, and any pod calling an external API.

Cost-Sensitive Resources (Teardown Targets)

Resource	Rough cost	Note
NAT Gateway	~\$0.045/hr (~\$32/mo) + data	First ongoing cost; single NAT chosen to halve it
EKS control plane	~\$0.10/hr	Destroy the cluster nightly
GPU node (g6.xlarge)	~\$0.80/hr	The big one — never leave running overnight
RDS instance	varies by size	Small instance for dev

Discipline: terraform destroy (or targeted teardown) at the end of every session so no NAT, cluster, or GPU node bills overnight.

Design decision: one NAT gateway (not one-per-AZ) to reduce cost; a production deployment would use one NAT per AZ for full resilience.